

Ejemplo: torneo suizo de 4 rondas. Ana recibe un *bye* aleatorio en la ronda 2.

Ronda	Rival / resultado
-------	-------------------

1	Gana a Bruno
2	Bye aleatorio
3	Pierde contra Carla
4	Gana a Diego

Su récord mostrado sería **3-1-0** y tendría **9 puntos**: el *bye* le da los 3 puntos de ronda.

Pero para su **Opponent's Win %**, TOM solo promedia a los rivales que sí jugó: Bruno, Carla y Diego. El *bye* no aporta un rival ficticio ni cuenta como una victoria de Ana para este desempate.

Si al terminar el torneo los porcentajes de esos rivales son:

- Bruno: 50 %
- Carla: 75 %
- Diego: 25 % (aplicando el mínimo de 25 %)

Entonces:

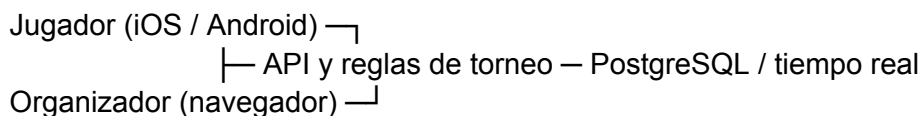
Op Win % de Ana = (50 % + 75 % + 25 %) / 3 = 50 %

No sería (50 + 75 + 25 + 100) / 4; ese 100 % por el *bye* no existe en el cálculo. El **Op Op Win %** se calcula después con los rivales reales de Bruno, Carla y Diego; igualmente, el *bye* no introduce un oponente ficticio.

Te recomiendo este stack:

- App de jugadores: **React Native + Expo**
- Panel de organizador y portal público: **Next.js**
- Lenguaje compartido: **TypeScript**
- Backend: **Supabase** (PostgreSQL, autenticación, archivos y tiempo real)
- Lógica crítica de torneos: **funciones de servidor** —no en el teléfono— para generar pareos, registrar resultados, recalcular standings y evitar dobles reportes.
- Notificaciones: **Expo Push Notifications** para avisar “ya están los pareos de la ronda 3”.
- Despliegue: **Vercel** para el panel web y **Supabase** para base de datos/backend.

La clave es que ambos clientes usan el mismo backend y modelo de datos:



No intentaría obligar al organizador a usar la misma interfaz que el jugador. Comparte componentes, autenticación, tipos y lógica de negocio, pero crea dos experiencias:

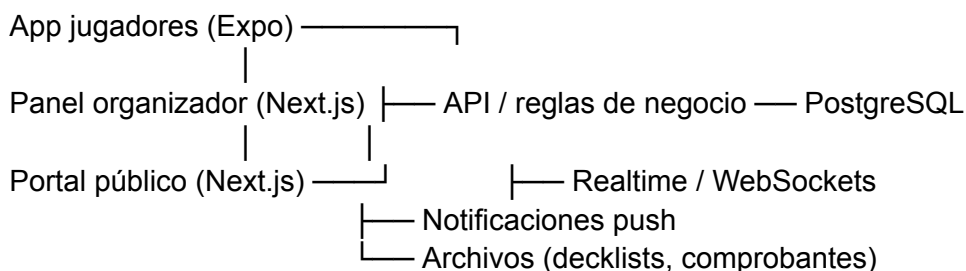
- Jugador: registro, *check-in*, pareja/mesa, contador de ronda, reporte y standings.

- Organizador: alta de torneo, lista de inscritos, aprobación, generación de pareos, corrección de resultados, impresión de hojas y auditoría.

Para una primera versión, usaría una **PWA instalable** para el panel web: funciona muy bien en laptop y también se puede abrir desde el móvil del organizador sin requerir una segunda app. Next.js permite construir una PWA con instalación y notificaciones web; Expo permite llevar la app de jugadores a iOS, Android y también web si lo necesitas. [Next.js PWA](#), [Expo para web](#), [Supabase Realtime](#)

Para el algoritmo de pareo, modelaría desde el inicio: historial de enfrentamientos, *byes*, *drops*, mesas, rondas bloqueadas y una bitácora inmutable de cambios. Eso será mucho más importante que el framework visual: un pareo incorrecto destruye la confianza de los jugadores.

Usaría una arquitectura de **monolito modular con backend centralizado y actualizaciones en tiempo real**. Es la mejor relación entre velocidad de desarrollo, fiabilidad y capacidad de crecer para una plataforma de torneos.



La idea central es esta:

- **Clientes separados por experiencia**, no por datos:
 - App móvil para jugadores.
 - Panel web responsivo para organizadores, que también abre bien en celular.
 - Página pública para consultar torneos, standings y pareos sin iniciar sesión, si así lo decides.
- **Un único núcleo de negocio en el servidor:**
 - Crear torneo, registrar jugadores y validar *check-in*.
 - Generar pareos suizos.
 - Bloquear y publicar una ronda.
 - Recibir y confirmar resultados.
 - Recalcular standings, desempates y *top cut*.
 - Registrar quién modificó qué y cuándo.
- **Base de datos PostgreSQL** como fuente única de verdad. No permitas que el teléfono “decida” pareos o standings: el cliente solo muestra información y solicita acciones; el servidor valida y confirma.
- **Tiempo real** para que, al publicar una ronda, el organizador, las pantallas públicas y los jugadores reciban el cambio al instante. Supabase Realtime puede emitir cambios de la base de datos. [Documentación](#)

La organizaría en módulos independientes dentro del mismo backend:

Módulo	Responsabilidad
Identidad	Cuentas, roles, tiendas y permisos
Torneos	Configuración, formatos, rondas y cortes
Pareos	Algoritmo suizo, <i>byes</i> , no repetir rivales y mesas
Resultados	Reporte, confirmación, disputas y correcciones

Standings	Puntos, Op Win %, Op Op Win % y criterios por juego
Notificaciones	Pareos publicados, recordatorios y cambios
Auditoría	Historial inalterable de acciones críticas

Para el primer producto no usaría microservicios. Separar pareos, notificaciones y usuarios en servicios distintos añade complejidad operativa sin aportar mucho. Cuando tengas muchos eventos simultáneos, podrías extraer el módulo de pareos o notificaciones a procesos independientes, sin cambiar la experiencia de los usuarios.

El punto más delicado es la publicación de ronda: debe ser una **operación atómica**. O se generan, guardan y publican todos los pareos correctamente, o no se publica nada. Así evitas que un jugador vea una ronda a medias o que dos personas reporten resultados incompatibles.